

- The total number of available signals, and default action
- Signals useful for our purpose
- Priority available for signals
- Presence of real-time signals
- Inter-process communications
 - Communication between related/unrelated processes
 - Inter-thread communication
 - Shared memory set-up and usage
 - Synchronising critical sections
 - Thread synchronisation
 - Priority inheritance support
 - Priority-based waiter queue
 - Deadlock detection tool
 - Communication between different systems
 - Sender-receiver synchronisation if one of them is faster (blocking calls)
 - Error-checking in sender/receiver: if the receiver or sender fails, and an exception or buffer overflow occurs, how does one prevent the failure from spreading? (single point of failure)
 - How to handle a graceful exit or shut-down—unwinding operations before exit
 - Scenarios in which two IPCs will be working together (shared memory/message queue—semaphore)
 - Need for event-driven programming (message notification through signal or creation of a new process or thread, wait queue, completion)
 - Shared re-entrant code between processes (shared read-only portion; for example, text portion of the vi editor, shared libraries)
 - Reducing kernel overhead in IPC mechanisms
 - Deriving a real-time response from IPC mechanisms (priority setting, time-out, light-weight IPC, priority inheritance support and priority waiter queue)
 - Self-correction systems (periodically cleaning unused PIDs, file descriptors, IPC ids, *malloc* pointers via *cron* jobs)

Network management

- We should be able to access/communicate with other systems, when the platform is running in a distributed environment, using *telnet* (teletype network) or *ssh* (secure shell), or through a Web server.
 - Does the platform have support for remote file access, remote debugging and remote updating?
 - Does the platform support protocols like TCP and SNMP?
 - Does the platform have tools support for monitoring, regulating and blocking traffic on the network?
 - Does the platform have enough tools to monitor memory and performance, such as *Purify* and *Quantify*? These tools are of great help during integration and systems testing. Architects must validate important test cases during system and integration testing.
- Does the platform support enough tests to simulate security threats that will be seen in the actual network? Is the platform robust enough to ward off security threats?

Performance tuning

- What are the code optimisation techniques and tools available?
- How could we reduce context switch latency (pinning to a processor)?
- How do we fine-tune the timer interrupt period for the desktop, server and real-time systems?
- Do we have a provision for a tick-less kernel?
- What performance analysis and monitoring tools are available?
- How to override system limitations?
- What levels of debugging support are available:
 - user level
 - kernel level
 - monitoring tools
 - tracing tools
 - kernel crash dump analysis

Support for standards and forum specifications

The operating system of choice should have compliance for standards/forum specifications, such as:

- Institute of Electrical and Electronics Engineers (IEEE) [5]
- The Open Group [6]
- Portable Operating System Interface [for Unix] (POSIX) [7]
- Linux Standard Base (LSB) specifications [8]
- Embedded Linux Consortium (ELC) platform specifications [9]



RHCE | RHCSS



CCNA



SOLARIS



ORACLE DBA

RHCE +

Virtualization

- Pioneers in LINUX Training
- Certified Faculties
- Corporate Clients
-DELL, NRS&A, GENPACT ETC.,
- Instructor Led Training
- 1:1 Systems/Routers
- Weekend Batches
- Placement Assistance

New batches
07, 14, 21, 28, June 2010



Amrita Technologies

844/1, Shantiniketan Colony, Mahindra Hills,
East Marredpally, Secunderabad-26.

Tel: 9393733174 | 27733174 | 27737969
Email: aict.hybd@amrita.ac.in | www.amritahyd.org

